ACTIVE DIRECTORY LAB | WEEK 7 MINI-PROJECT

Automating user provisioning with one PowerShell script

Create, enable, group, report. Run it twice, then try to break it.

Fictional users. No passwords in the file. Sensitive details masked.

THE GOAL

From manual commands to one script

This week I did everything by hand: OUs, users, passwords, groups. The mini-project automates it.

- Start from a list of employee records
- Build each login and OU path from the record
- Create the account only if it does not exist
- Enable it with a password that is never stored in the file
- Add it to its department group
- Print a summary table

THE DESIGN

What happens for each employee

1. Read the record
first name, last name, department

2. Build login and OU path
firstname.lastname, OU=Dept,DC=cyber-lab,DC=local

3. Create if missing
Get-ADUser first, New-ADUser only if absent

4. Enable if needed
ask for the password only when required

5. Add to the team group
skip if already a member

6. Add a row to the summary
printed as a table at the end

PREP

Fixing copy-paste into the VM

I could not paste code into the VM. The VirtualBox clipboard setting needs the Guest Additions installed inside the VM. The first check returned nothing, the second shows the service running after the install.

Before: nothing. After: Running

```
Get-Service VBox*
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka
.ms/PSWindows

PS C:\Users\Administrator> Get-Service VBox*
PS C:\Users\Administrator>
```

```
PS C:\Users\Administrator> Get-Service VBox*

Status   Name               DisplayName
------   ----               -----------
Running  VBoxService        VirtualBox Guest Additions Service
```

STEPS 1 TO 3

A list of records, a loop, then login and path

The script grew one idea at a time. The capture shows three successive versions: first employee, loop, then login and OU path built from each record.

Same login format for everyone

```
foreach ($emp in $employees) {
$login = ("$($emp.First).$($emp.Last)").ToLower()
$ouPath = "OU=$($emp.Dept),$domainPath"
```

```
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
First employee: Ibrahim Diallo
PS C:\Users\Administrator> notepad C:\Labs\provision-ad.ps1
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Employee: Ibrahim Diallo - Department: Marketing
Employee: David Laporte - Department: Logistics
Employee: Andre Dante - Department: IT
PS C:\Users\Administrator> notepad C:\Labs\provision-ad.ps1
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Login: ibrahim.diallo - Path: OU=Marketing,DC=cyber-lab,DC=local
Login: david.laporte - Path: OU=Logistics,DC=cyber-lab,DC=local
Login: andre.dante - Path: OU=IT,DC=cyber-lab,DC=local
PS C:\Users\Administrator>
```

STEP 4

Creating the accounts

New-ADUser with variables instead of typed values. New accounts are disabled by default.

Enabled: False

```
New-ADUser -Name "$($emp.First) $($emp.Last)" -GivenName $emp.First
-Surname $emp.Last -SamAccountName $login -Path $ouPath
```

```
Name              : Ibrahim Diallo
Enabled           : False
DistinguishedName : CN=Ibrahim Diallo,OU=Marketing,DC=cyber-lab,DC=local

Name              : David Laporte
Enabled           : False
DistinguishedName : CN=David Laporte,OU=Logistics,DC=cyber-lab,DC=local

Name              : Andre Dante
Enabled           : False
DistinguishedName : CN=Andre Dante,OU=IT,DC=cyber-lab,DC=local
```

STEP 5

Making it safe to run twice

Before creating an account, the script asks the directory if it exists. No error on a second run.

Re-runnable

```
$exists = Get-ADUser -Filter "SamAccountName -eq '$login'"
if ($exists) { ... } else { New-ADUser ... }
```

```
PS C:\Users\Administrator> notepad C:\Labs\provision-ad.ps1
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Already exists, skipped: ibrahim.diallo
Already exists, skipped: david.laporte
Already exists, skipped: andre.dante
```

STEPS 6 AND 7

A password the script never stores

Anyone who opens the script sees everything written in it. So the password is asked at run time and kept as a SecureString.

```
$password = Read-Host "Enter the initial password for the accounts"
-AsSecureString
Set-ADAccountPassword -Identity $login -Reset -NewPassword $password
Enable-ADAccount -Identity $login
```

- The password is set first, then the account is enabled.
- Typed characters never appear on screen.

STEP 7

All three accounts enabled

Each account sits in its own OU and Enabled is True.

Enabled: True

```
Name              : Ibrahim Diallo
Enabled           : True
DistinguishedName : CN=Ibrahim Diallo,OU=Marketing,DC=cyber-lab,DC=local

Name              : David Laporte
Enabled           : True
DistinguishedName : CN=David Laporte,OU=Logistics,DC=cyber-lab,DC=local

Name              : Andre Dante
Enabled           : True
DistinguishedName : CN=Andre Dante,OU=IT,DC=cyber-lab,DC=local
```

STEP 8

Adding each employee to the team group

The group name is built from the department. The line with the typed password is masked on this capture.

Added to group

```
$group = "$($emp.Dept)-team"
Add-ADGroupMember -Identity $group -Members $login
```

```
PS C:\Users\Administrator> notepad C:\Labs\provision-ad.ps1
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Enter the initial password for the accounts: [masked]
Password received (hidden).
Employees in the list: 3
Already exists: ibrahim.diallo
Added to group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Added to group: david.laporte -> Logistics-team
Already exists: andre.dante
Added to group: andre.dante -> IT-team
PS C:\Users\Administrator>
```

RUN IT TWICE

Second run: nothing breaks

Every line is Already exists or Already in group, and there is no red error.

Idempotent

```
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Enter the initial password for the accounts:
Password received (hidden).
Employees in the list: 3
Already exists: ibrahim.diallo
Already in group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Already in group: david.laporte -> Logistics-team
Already exists: andre.dante
Already in group: andre.dante -> IT-team
PS C:\Users\Administrator>
```

TESTING

Two flaws I found by running it again

Reading my own output, I noticed the script asked for a password even when nothing needed one, and accepted an empty one silently.

- Flaw 1: password asked at the start, even if every account was already enabled.
- Flaw 2: pressing Enter on an empty password did nothing, with no message.
- Fix: ask only when a disabled account needs it, and refuse an empty password with a warning.

FIX 1

No password asked when none is needed

All three accounts were already enabled, so the script ran straight through.

No useless prompt

```
if ($null -eq $password) { $password = Read-Host ... -AsSecureString }
```

```
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Already exists: ibrahim.diallo
Already in group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Already in group: david.laporte -> Logistics-team
Already exists: andre.dante
Already in group: andre.dante -> IT-team
PS C:\Users\Administrator>
```

FIX 2

An empty password is refused

I disabled one account on purpose to test. Pressing Enter on the prompt gives a warning, and the account stays disabled.

Warning shown

```
if ($password.Length -eq 0) { Write-Warning "Empty password: $login
was not enabled." }
```

```
PS C:\Users\Administrator> Disable-ADAccount -Identity ibrahim.diallo
>> Get-ADUser -Identity ibrahim.diallo | Select-Object Name, Enabled | Forma
t-List


Name    : Ibrahim Diallo
Enabled : False



PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Already exists: ibrahim.diallo
Enter the initial password for the accounts:
WARNING: Empty password: ibrahim.diallo was not enabled.
Already in group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Already in group: david.laporte -> Logistics-team
Already exists: andre.dante
Already in group: andre.dante -> IT-team
PS C:\Users\Administrator>
```

FIX 2, THEN SUCCESS

A real password: account enabled

Second run with a valid password. Enabled: ibrahim.diallo, and the check returns True. Password line masked.

Enabled: True

```
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Already exists: ibrahim.diallo
Enter the initial password for the accounts: [masked]
Enabled: ibrahim.diallo
Already in group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Already in group: david.laporte -> Logistics-team
Already exists: andre.dante
Already in group: andre.dante -> IT-team
PS C:\Users\Administrator> Get-ADUser -Identity ibrahim.diallo | Select-Obje
ct Name, Enabled | Format-List


Name    : Ibrahim Diallo
Enabled : True
```

RESULT

The summary table

One row per employee: login, department, enabled state and group.

3 employees provisioned

```
PS C:\Users\Administrator> C:\Labs\provision-ad.ps1
Employees in the list: 3
Already exists: ibrahim.diallo
Already in group: ibrahim.diallo -> Marketing-team
Already exists: david.laporte
Already in group: david.laporte -> Logistics-team
Already exists: andre.dante
Already in group: andre.dante -> IT-team

Summary:

Login          Department Enabled Group
-----          ---------- ------- -----
ibrahim.diallo Marketing     True Marketing-team
david.laporte  Logistics     True Logistics-team
andre.dante    IT            True IT-team
```

THE FINAL SCRIPT (1/2)

provision-ad.ps1

```
# provision-ad.ps1
# Creates fictional employees in Active Directory (Week 7 mini-project)

$employees = @(
    @{ First = "Ibrahim"; Last = "Diallo";  Dept = "Marketing" },
    @{ First = "David";   Last = "Laporte"; Dept = "Logistics" },
    @{ First = "Andre";   Last = "Dante";   Dept = "IT" }
)

$domainPath = "DC=cyber-lab,DC=local"
$password = $null
$report = @()

Write-Host "Employees in the list:" $employees.Count

foreach ($emp in $employees) {
    $login = ("$($emp.First).$($emp.Last)").ToLower()
    $ouPath = "OU=$($emp.Dept),$domainPath"

    $exists = Get-ADUser -Filter "SamAccountName -eq '$login'"

    if ($exists) {
        Write-Host "Already exists:" $login
    }
    else {
        New-ADUser -Name "$($emp.First) $($emp.Last)" -GivenName $emp.First -Surname
            $emp.Last -SamAccountName $login -Path $ouPath
        Write-Host "Created:" $login "in" $ouPath
    }
```

THE FINAL SCRIPT (2/2)

provision-ad.ps1

```
    $user = Get-ADUser -Identity $login
    if (-not $user.Enabled) {
        if ($null -eq $password) {
            $password = Read-Host "Enter the initial password for the accounts"
                -AsSecureString
        }
        if ($password.Length -eq 0) {
            Write-Warning "Empty password: $login was not enabled."
            $password = $null
        }
        else {
            Set-ADAccountPassword -Identity $login -Reset -NewPassword $password
            Enable-ADAccount -Identity $login
            Write-Host "Enabled:" $login
        }
    }

    $group = "$($emp.Dept)-team"
    $members = Get-ADGroupMember -Identity $group | Select-Object -ExpandProperty
        SamAccountName
    if ($members -notcontains $login) {
        Add-ADGroupMember -Identity $group -Members $login
        Write-Host "Added to group:" $login "->" $group
    }
    else {
        Write-Host "Already in group:" $login "->" $group
    }

    $final = Get-ADUser -Identity $login
    $report += [PSCustomObject]@{
        Login      = $login
        Department = $emp.Dept
        Enabled    = $final.Enabled
        Group      = $group
    }
}

Write-Host ""
Write-Host "Summary:"
$report | Format-Table -AutoSize
```

HONEST LIMITS

What this script does not do yet

- The employee list is written inside the script. Next: read it from a CSV file.
- It assumes the OUs and the groups already exist.
- Everyone gets the same initial password. Acceptable in a lab only.
- No log file: the screen output is the only record.

TAKEAWAYS

What I learned

- A script that works once is not finished: run it twice, then try to break it.
- Check if something exists before creating it.
- Never store a password in the file. Ask at run time.
- Refuse invalid input and say why.
- Build in small steps and verify each one.

QUESTION

What is the first bug you found by re-running your own script?